DevOps Explanation Request

Alright vicky 😈
DAY-26: Kubernetes Network Policies — this is where *"it works"* turns into *"it's secure"*.

Same flow, same clarity 👇
👉 **way → problem → how Kubernetes solves it → hands-on → verification → traps**

☠️ Way (Default Kubernetes Networking)

By default in Kubernetes:

- **Any Pod can talk to any other Pod**
- **Any Pod can talk to any Service**
- No firewall between workloads

So this is allowed 👇

```scss

frontend → backend  ✅ (needed)
frontend → database ✅ (NOT needed)
random pod → database ✅ (VERY BAD)
```

Kubernetes networking is **flat by default**.

💣 Problem

In real systems:

- Frontend **should not** talk to DB directly

- Only backend **should** access DB
- If one pod is compromised → attacker can move everywhere (lateral movement)

This violates:

- Least privilege
- Zero trust
- Basic security hygiene

🛠️ How Kubernetes Solves This

Using **Network Policies**

> **NetworkPolicy = firewall rules for Pods**

Important:

- Kubernetes only defines the **API**
- The **CNI plugin enforces it**

Step 1 – What is CNI and why we need it?

CNI (Container Network Interface) is responsible for:

- Pod-to-Pod networking
- IP assignment
- Enforcing NetworkPolicy (if supported)

Common CNI providers

- Weave Net
- Flannel
- Calico ✅
- Cilium

⚠️ Not every CNI supports NetworkPolicy

CNI	NetworkPolicy
kind-net	❌
Flannel	❌
Calico	✅
Cilium	✅

That's why you **must choose CNI carefully**.

CNI runs as DaemonSet

- One CNI pod **per node**
- Manages networking for pods on that node

This is why you'll always see:

```bash
kubectl get pods -n kube-system
```

→ one Calico pod per node.

Step 2 – Setting up KIND with Calico

You disabled default CNI 👇 (important)

```yaml
networking:
  disableDefaultCNI: true
```

```
podSubnet: 192.168.0.0/16
```

Why?

- `kind-net` does NOT support NetworkPolicy
- You want **Calico** to take control

Good decision 👍

Step 3 – Application Setup (Before NetworkPolicy)

You deployed:

- frontend Pod + Service
- backend Pod + Service
- mysql Pod + Service (db)

At this stage:

```bash
frontend → backend → mysql
frontend → mysql ❌ (but actually works)
```

Everything can talk to everything.

This is the **problem state**.

Step 4 – NetworkPolicy basics (VERY IMPORTANT)

A NetworkPolicy has 3 core parts:

```csharp
```

```
podSelector → WHICH pods are protected
policyTypes → Ingress / Egress
rules       → WHO can talk + on WHICH ports
```

Key rule to remember:

> **The moment a pod is selected by a NetworkPolicy,**
> **all traffic is DENIED by default except what you allow.**

This is critical.

Step 5 – Your NetworkPolicy (Hands-on)

`net-policy.yaml`

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-test
spec:
  podSelector:
    matchLabels:
      name: mysql
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          role: backend
```

```yaml
  ports:
  - port: 3306
```

Let’s decode this line-by-line 🧠

`podSelector`

```yaml
matchLabels:
  name: mysql
```

➡ This policy **applies to mysql pods only**

`policyTypes: Ingress`

- We are controlling **incoming traffic**
- Outgoing traffic from mysql is untouched

`ingress.from`

```yaml
from:
- podSelector:
    matchLabels:
      role: backend
```

➡ Only pods with label `role=backend` are allowed

`ports`

```yaml
ports:
- port: 3306
```

➡ Only MySQL port is allowed

Step 6 – What traffic is allowed vs blocked

Allowed ✅

```nginx
backend → mysql:3306
```

Blocked ❌

```cpp
frontend → mysql
any-other-pod → mysql
```

Even if:

- Service exists
- DNS resolves
- Pod IP is reachable

👉 NetworkPolicy drops packets at CNI level.

Step 7 – Verify behavior (VERY IMPORTANT)

Before NetworkPolicy:

```bash
kubectl exec frontend -- curl db:3306
# works
```

After NetworkPolicy:

```bash
kubectl exec frontend -- curl db:3306
# ❌ connection timeout
```

From backend:

```bash
kubectl exec backend -- curl db:3306
# ✅ works
```

This confirms:

- Policy is enforced
- CNI is working
- Labels matter

Step 8 – Key rules people forget (INTERVIEW GOLD)

1️⃣ NetworkPolicy is deny-by-default

Once applied to a pod.

2️⃣ Policies are additive

Multiple policies can apply to same pod.

3️⃣ Services do NOT bypass NetworkPolicy

Policies apply to **Pod IPs**, not Services.

4️⃣ DNS must be explicitly allowed (sometimes)

If you restrict egress heavily, pods can't resolve names.

Step 9 – Common mistakes (seen DAILY)

❌ Using Flannel / kind-net and expecting NetworkPolicy
❌ Forgetting labels don't match
❌ Thinking Service name matters
❌ Blocking DNS accidentally
❌ Expecting NetworkPolicy to control NodePort / LB

Step 10 – Final Mental Model (lock this)

```pgsql
CNI provides networking
NetworkPolicy provides rules
Labels decide traffic
No policy = allow all
Policy exists = deny unless allowed
```

Or even simpler:

> **NetworkPolicy = Kubernetes firewall for pods enforced by the CNI plugin**

If you want next 😏🔥

- Egress NetworkPolicy
- Default-deny policy pattern
- DNS allow-list policy
- Calico vs Cilium differences
- CKA NetworkPolicy traps

Say the word.